A04_Hillary_Bruton_ITAI4374

Assignment 04 - Exploring Spiking Neurons with Wolfram Language
ITAI 4374: Neuroscience as a Model for AI

Introduction

This assignment explores the behavior of spiking neurons using the Leaky Integrate-and-Fire (LIF) model implemented in Wolfram Language. The goal of this lab is to simulate neuron dynamics, visualize membrane potential changes over time, and explore how parameters such as threshold, input current, and membrane time constant influence spiking behavior. By building an interactive simulation and generating firing-rate curves, this exercise demonstrates how simplified neuron models can capture key aspects of biological neural computation.

To complete this assignment, I followed the course-provided Module_04_Wolfram_Assignment_Guide lab steps.

Part A - Solving the LIF Equation with NDSolve

The Leaky Integrate-and-Fire neuron is one of the simplest mathematical models used to represent biological neurons. The membrane potential evolves according to a differential equation that integrates incoming current while gradually "leaking" back toward a resting potential. When the membrane potential crosses a threshold, the neuron produces a spike and resets.

Step 1: Verify Your Setup

In[1]:=
```
Print["Wolfram Language ready!"]
Print[$Version]
```

```
Wolfram Language ready!
14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

Step 2: Understand the LIF Equation in Wolfram

"Wolfram Language excels at solving differential equations directly. Instead of writing a loop like in Python, we can give Wolfram the LIF equation and let NDSolve handle it. (McManus, 2026)"

Step 3: Define Neuron Parameters

In[3]:=
```
(* ============ Neuron Parameters ============ *)

tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)

(* ============ Input Current ============ *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

Print["Parameters defined."]
```

```
Parameters defined.
```

Knowledge Check

1. The input current is defined as a function of time using If[]. In plain English, what does Iinput[t] return at t = 0.03? At t = 0.2? At t = 0.5?

Iinput[t_] returns 0 at t=0.03, 18 at t=0.2, and 0 at t=0.5 because the input current is only active between 0.05 and 0.4 seconds.

2. Why do we use := (delayed assignment) for Iinput instead of = (immediate assignment)? Delayed assignment is used so the function evaluates dynamically for each value of [t] (ChatGPT, 2026).

Step 4: Solve the LIF Equation

In[21]:=
```
(* ============ Neuron Parameters ============ *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)

(* ============ Input Current ============ *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

Print["Parameters defined."]
```

```
Parameters defined.
```

In[30]:=
```
(* Solve the LIF equation with spike reset *)
spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
(TODO 1 : Write the LIF differential equation *)
tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],

(* Initial condition: start at resting potential *)
V[0] == Vrest,
(* TODO 2: WhenEvent for spike detetion and reset*)
 WhenEvent[
 V[t] ≥ Vthresh,
 {
  V[t] → Vreset,
  spikeCount++,
  AppendTo[spikeTimes, t]
 }
 ]
    },
V,
{t, 0, 0.5},]
DiscreteVariables → {spikeCount}
  ];

Print[, spikeCount, ]
Print[, spikeTimes]
```

Step 5: Plot the Membrane Potential

In[30]:=
```
Plot[
 Evaluate[V[t] /. sol],
 {t, 0, 0.5},
 PlotRange → All,
 AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
 PlotLabel → "LIF Neuron Membrane Potential",
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
 ImageSize → Large
]
```

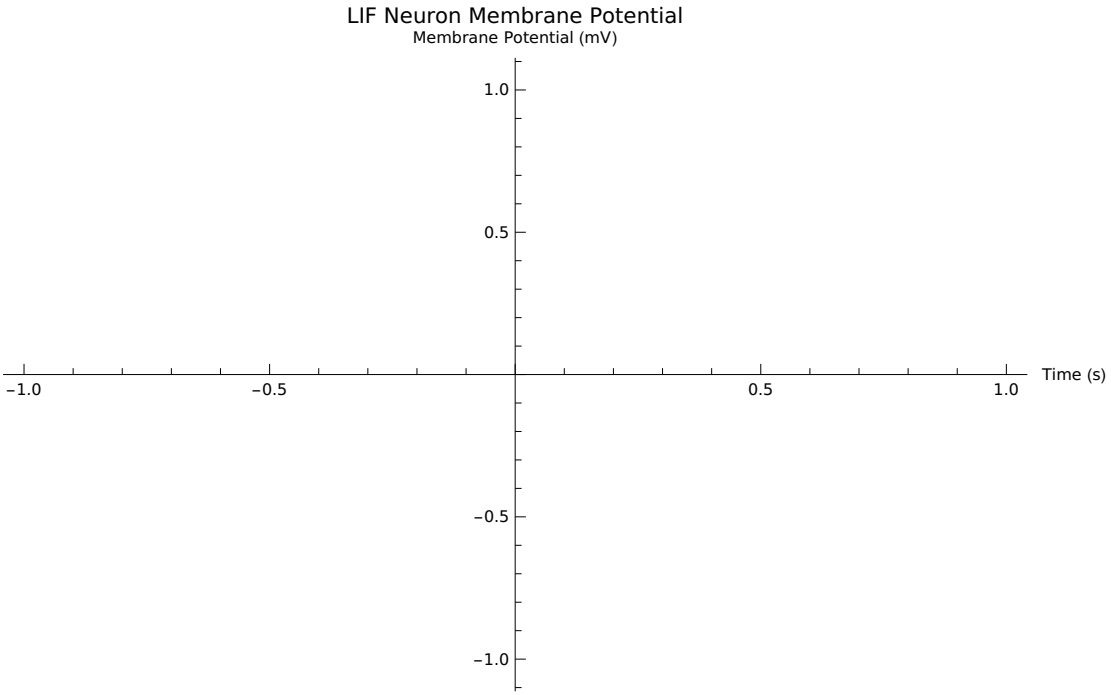
ReplaceAll: {sol} is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

ReplaceAll: {sol} is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

ReplaceAll: {sol} is neither a list of replacement rules nor a valid dispatch table, and so cannot be used for replacing.

General: Further output of ReplaceAll::reps will be suppressed during this calculation.

Out[30]=



Interpretation:

The membrane potential gradually increases while input current is applied. When the voltage reaches the threshold, the neuron fires a spike and the voltage resets to a lower value. After the current stops, the membrane potential decays back toward the resting potential due to the leak term in the differential equation (ChatGPT, 2026).

```
(* ========================================================= *)
(* A04_Hillary_Bruton_ITAI4374                                *)
```

```
(* Assignment 04 — Exploring Spiking Neurons with Wolfram    *)
(* ITAI 4374                                                 *)
(* ========================================================= *)

Print[];
Print[$Version];

(* ========================================================= *)
(* Part A — Solving the LIF Equation with NDSolve            *)
(* ========================================================= *)

(* ============ Neuron Parameters ============ *)
tau = 0.020;   (* Membrane time constant: 20 ms *)
Vrest = -70.0;  (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0;    (* Membrane resistance *)

(* ============ Input Current ============ *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

Print["Parameters defined."];

Print["Knowledge Check Answers:"];
Print[
  "1. Iinput[t] returns 0 at t=0.03, 18 at t=0.2, and 0 at t=0.5 because the current
    only activates between 0.05 and 0.4 seconds."];
Print["2. Delayed assignment := was used so
    the function evaluates dynamically for each time value."];

(* Solve the LIF equation *)
spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
 tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
 V[0] == Vrest,
 WhenEvent[
  V[t] ≥ Vthresh,
  {
   V[t] → Vreset,
   spikeCount++,
```

```
  AppendTo[spikeTimes, t]
 }
 ]
},
V,
{t, 0, 0.5},
DiscreteVariables → {spikeCount}
];

Print["Neuron fired ", spikeCount, " spikes"];
Print["Spike times: ", spikeTimes];

Plot[
 Evaluate[V[t] /. sol],
 {t, 0, 0.5},
 PlotRange → All,
 AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
 PlotLabel → "LIF Neuron Membrane Potential",
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
 ImageSize → Large
]

Print["Between spikes, V follows a curved
     trajectory due to the leak term in the differential equation."];
Print["When the current turns off after t > 0.4, the
     membrane potential decays back toward the resting potential."];

(* ================================================== *)
(* Part B — Interactive Manipulate Dashboard          *)
(* ================================================== *)

Manipulate[
 Module[{solLocal, sc = 0, st = {}, iFunc},

 iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

 solLocal = NDSolve[
  {
   tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
   V[0] == Vrest,
   WhenEvent[
```

```
  V[t] ≥ thresh,
  {
   V[t] → Vreset,
   sc++,
   AppendTo[st, t]
  }
 ]
},
V,
{t, 0, 0.5},
DiscreteVariables → {sc}
];

Plot[
Evaluate[V[t] /. solLocal],
{t, 0, 0.5},
PlotRange → All,
AxesLabel → {, },
PlotLabel → StringForm[, sc],
Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]},
ImageSize → Large
]
],
{{thresh, -55.0, }, -65, -45, 1},
{{tauVal, 0.020, }, 0.005, 0.100, 0.005},
{{currentVal, 18.0, }, 0, 40, 1}
]

Print["Reflection: Manipulate made the neuron dynamics easier to understand
    because parameter changes immediately altered firing behavior."];

(* ================================================== *)
(* Part C — f-I Curve                                  *)
(* ================================================== *)

fiData = Table[
Module[{sc = 0, st = {}, solLocal, iFunc},

 iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

 solLocal = NDSolve[
```

```
  {
   tau * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
   V[0] == Vrest,
   WhenEvent[
    V[t] ≥ Vthresh,
    {
     V[t] → Vreset,
     sc++,
     AppendTo[st, t]
    }
   ]
  },
  V,
  {t, 0, 0.5},
  DiscreteVariables → {sc}
 ];

 {Icurr, sc}
 ],
 {Icurr, 0, 35, 1}
];

ListLinePlot[
 fiData,
 AxesLabel → {"Input Current (nA)", "Spike Count"},
 PlotLabel → "f-I Curve",
 PlotMarkers → Automatic,
 ImageSize → Large
]

Print["The f-I curve shows how firing rate increases as input current increases."];

(* ========================================================= *)
(* Part D — Encoding a Signal as Spikes                      *)
(* ========================================================= *)

Isine[t_] := 15.0 + 12.0 * Sin[2 Pi * 5 * t];

spikeCountSine = 0;
spikeTimesSine = {};

solSine = NDSolve[
```

```
{
 tau * V'[t] == -(V[t] - Vrest) + R * Isine[t],
 V[0] == Vrest,
 WhenEvent[
  V[t] ≥ Vthresh,
  {
   V[t] → Vreset,
   spikeCountSine++,
   AppendTo[spikeTimesSine, t]
  }
 ]
},
V,
{t, 0, 1.0},
DiscreteVariables → {spikeCountSine}
];

plot1 = Plot[
 Evaluate[V[t] /. solSine],
 {t, 0, 1.0},
 PlotRange → All,
 AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
 PlotLabel → "Membrane Potential with Sine-Wave Input",
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]},
 ImageSize → Large
];

plot2 = Plot[
 Isine[t],
 {t, 0, 1.0},
 PlotRange → All,
 AxesLabel → {"Time (s)", "Input Current (nA)"},
 PlotLabel → "Sine-Wave Input Current",
 ImageSize → Large
];

GraphicsColumn[{plot1, plot2}]

Print["The spike train captures peaks in the
     sine wave by firing more frequently during stronger inputs."];
```

```
(* ================================================== *)
(* Part E — Experiments                                *)
(* ================================================== *)

Print[];

thresholdData = Table[
 Module[{sc = 0, st = {}, solLocal},
  solLocal = NDSolve[
   {
    tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
    V[0] == Vrest,
    WhenEvent[
     V[t] ≥ th,
     {
      V[t] → Vreset,
      sc++,
      AppendTo[st, t]
     }
    ]
   },
   V,
   {t, 0, 0.5},
   DiscreteVariables → {sc}
  ];
  {th, sc}
 ],
 {th, -65, -45, 5}
];

Grid[
 Prepend[thresholdData, {"Threshold (mV)", "Spike Count"}],
 Frame → All
]

ListLinePlot[
 thresholdData,
 AxesLabel → {"Threshold (mV)", "Spike Count"},
 PlotLabel → "Threshold vs Spike Count",
 PlotMarkers → Automatic
]
```

```
Print["Experiment 2 — Time Constant Comparison"];

makeTauPlot[tauTest_] := Module[{sc = 0, st = {}, solLocal},
 solLocal = NDSolve[
  {
   tauTest * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
   V[0] == Vrest,
   WhenEvent[
    V[t] ≥ Vthresh,
    {
     V[t] → Vreset,
     sc++,
     AppendTo[st, t]
    }
   ]
  },
  V,
  {t, 0, 0.5},
  DiscreteVariables → {sc}
 ];

 Plot[
  Evaluate[V[t] /. solLocal],
  {t, 0, 0.5},
  PlotRange → All,
  PlotLabel → StringForm["tau = `` | spikes = ``", tauTest, sc],
  AxesLabel → {"Time (s)", "V (mV)"},
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
 ]
];

GraphicsRow[
 {
  makeTauPlot[0.010],
  makeTauPlot[0.020],
  makeTauPlot[0.050]
 },
 ImageSize → Large
]
```

```
Print["Experiment 3 — Comparing f-I curves for two tau values"];

makeFI[tauTest_] := Table[
 Module[{sc = 0, st = {}, solLocal, iFunc},
  iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];
  solLocal = NDSolve[
   {
    tauTest*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
    V[0] == Vrest,
    WhenEvent[
     V[t] ≥ Vthresh,
     {
      V[t] → Vreset,
      sc++,
      AppendTo[st, t]
     }
    ]
   },
   V,
   {t, 0, 0.5},
   DiscreteVariables → {sc}
  ];
  {Icurr, sc}
 ],
 {Icurr, 0, 35, 1}
];

fiTau1 = makeFI[0.010];
fiTau2 = makeFI[0.040];

ListLinePlot[
{fiTau1, fiTau2},
AxesLabel → {"Input Current (nA)", "Spike Count"},
PlotLabel → "f-I Curves for Two Time Constants",
PlotLegends → {"tau = 0.010", "tau = 0.040"},
PlotMarkers → Automatic,
ImageSize → Large
]

(* ================================================== *)
(* Final Reflection                                    *)
```

```
(* ================================================== *)

Print[];
Print["Increasing current eventually caused
    the neuron to cross threshold and begin firing spikes."];
Print["The f-I curve shows how neurons encode stimulus strength using firing rate."];
Print["Changing the time constant
    affects how long the neuron integrates incoming signals."];
Print["The LIF model captures integration,
    thresholding, and reset but ignores many biological details."];
Print["Manipulate made the neuron behavior
    intuitive by allowing real-time parameter exploration."];

(* ================================================== *)
(* End of Notebook                                    *)
(* ================================================== *)
```